A <u>filesystem</u> is just an array of bytes.

How do we store data in it?

Let's divide the filesystem into chunks.

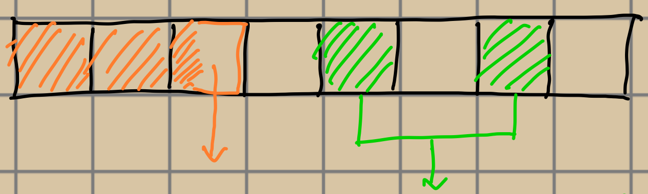
we call these chunks "blocks."

How big are the blocks? ⟶ 1 KB / 4 KB sound good.

4096 bytes

If we store a file in the fs, it may take up one or more blocks.

If a file takes up a block partially, we <u>cannot</u> put data of another file in it.

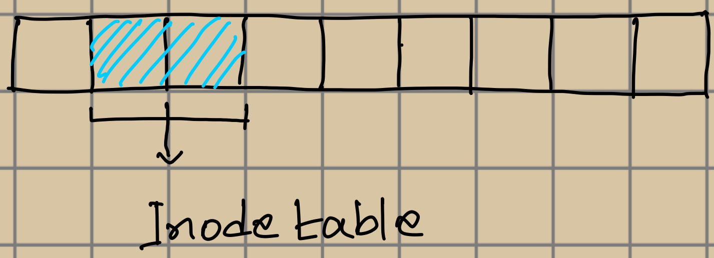


taken both are for a single file.

A file can take up contiguous <u>or</u> non-contiguous blocks. Thus, we need to know which blocks a file takes up, and in what order.

We use a structure that has the list of block indices for every file. We call this an "inode."

Inode
- type (enum)
- size (bytes)
- mode (perms)
- direct[ ] (block index)



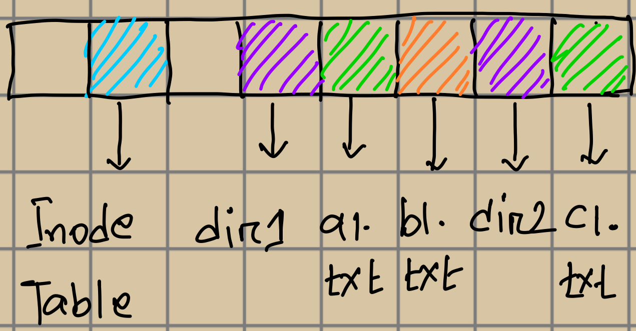
The direct[ ] array might not hold enough block indices. We use another pointer that points to a block that holds more data block indices.

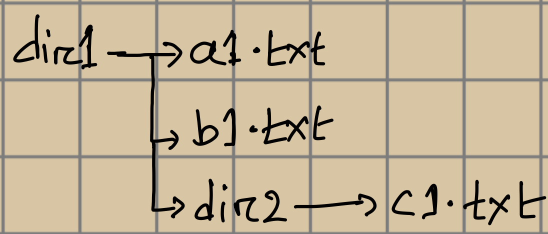
We might even have pointers to a block full of pointers that point to data blocks.

How do we store filenames and directories?

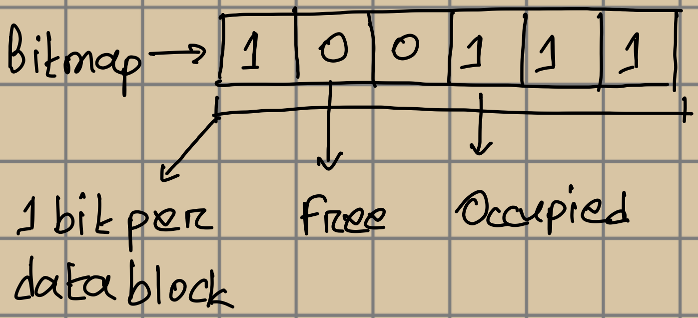
↳ Directories are also just files that hold filenames and inode numbers for those files.

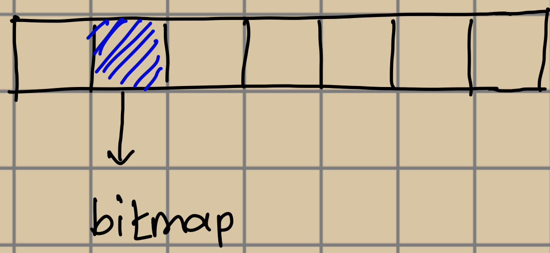
	Name	Inode
dir1	a1.txt	2
	b1.txt	3
	dir2	4
dir2	c1.txt	5



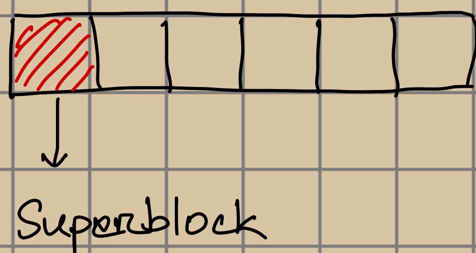


Files grow. So, we need to store which bytes are free.
We can use a "bitmap" to do that.





We also need to store all the properties of the fs in a dedicated block. Let's put it in index 0, and call it a "superblock." Let's also add a "magic number" in front of the superblock, to know that it is properly formatted.



Super block:

- magic number
- block size
- number of blocks in total
- number of inodes
- starting index of inode table
- number of blocks for inode table
- starting index of bitmap
- starting index of data blocks

Thus, the proposed layout: